

Orquestación multi-agente con sub-workflows

Pre-entrega Módulo 2 · Proyecto integrador Torque Motopartes
Autor: Enzo Saporiti · Cohorte 2026

1. Contexto y objetivo

Torque Motopartes es una tienda online (ficticia) de repuestos y accesorios para moto. En el Módulo 1 se construyó un agente único de atención al cliente que consultaba pedidos y respondía. En este checkpoint ese agente monolítico se descompone en una arquitectura **Manager-Worker**: un orquestador que clasifica la intención del cliente y delega cada tarea atómica en especialistas independientes, implementados como sub-workflows en lienzos separados de n8n.

Componentes de la arquitectura

Workflow	Rol	Tarea atómica	Usa IA
Torque - Manager (M2)	Orquestador	Recibe el mensaje, clasifica la intención en una taxonomía cerrada, delega, consolida y registra la trazabilidad.	Sí (clasificador, Claude Haiku 4.5)
Torque - Worker 1 Consulta Pedidos	Especialista	Busca el pedido en Google Sheets y verifica que el email coincida. Devuelve solo los datos necesarios.	No (costo cero de tokens)
Torque - Worker 2 Redacción	Especialista	Redacta la respuesta final al cliente a partir de los datos recibidos, sin inventar información.	Sí (Claude Haiku 4.5)

Flujo lógico

```
[Chat Trigger] -> [Router de Triage (Text Classifier + Claude Haiku 4.5)]
|
|- PEDIDOS_ENVIOS: [Contrato W1] -> [Delegar a Worker 1] -> [Contrato W2 - Pedidos] --+
|- DEVOLUCIONES_GARANTIAS: [Contrato W2 - Devoluciones] -----+
|- VENTAS: [Contrato W2 - Ventas] -----+
|
|                                     v
|                                     [Delegar a Worker 2] --+
|- OTRO / Other (fallback): [Contrato Escalamiento] -----+
+- Error en W1 o W2: [Contingencia Técnica] -----+
|
|                                     v
|                                     [Trazabilidad] -> [Log Observabilidad Slack] -> [Respuesta al cliente]
```

2. Workflow Manager (orquestador principal)

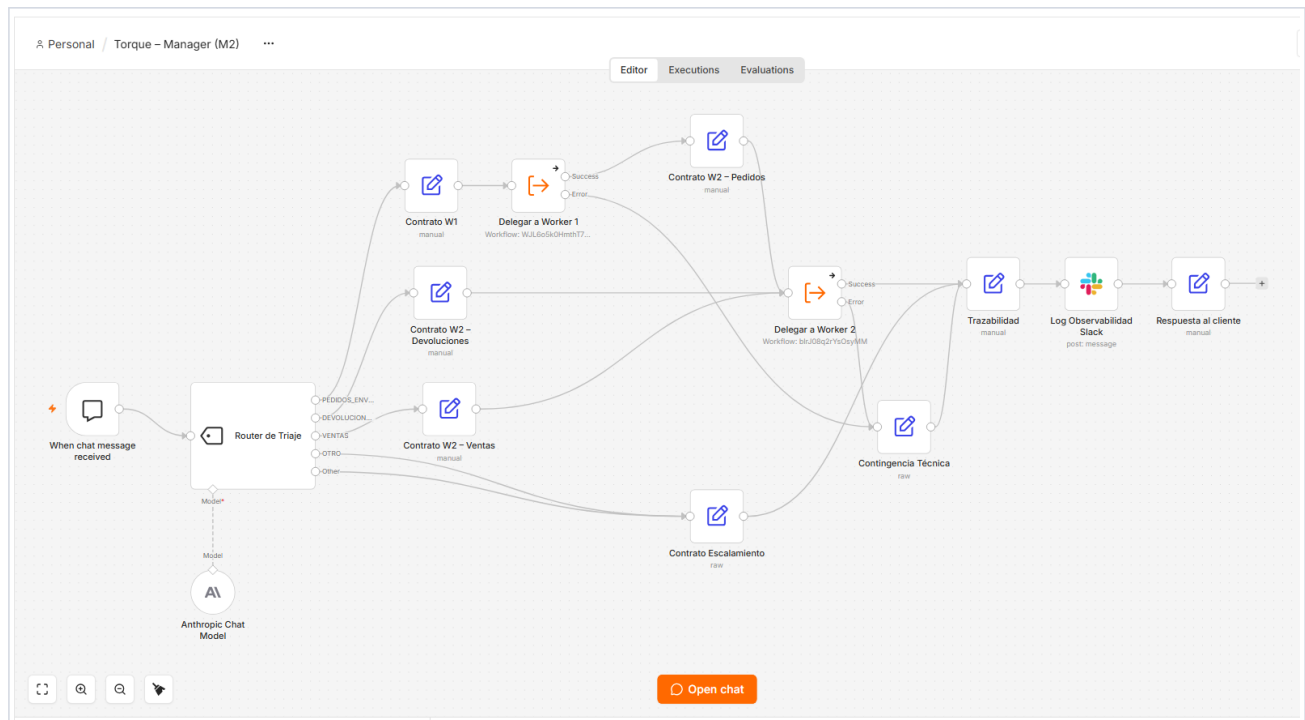


Figura 1. Lienzo completo del Manager, con los nodos Execute Workflow «Delegar a Worker 1» y «Delegar a Worker 2».

Nodo	Función
When chat message received	Trigger: captura el mensaje desestructurado del cliente.
Router de Triage	Clasificador semántico (Text Classifier) con taxonomía cerrada de 4 categorías y salida extra «Other» para intenciones dudosas.
Contrato W1 / Contrato W2 - ...	Nodos Set que limpian el payload y arman el contrato mínimo para cada Worker (antídoto del «Pasillo de la Muerte»).
Delegar a Worker 1 / 2	Execute Workflow con «Wait For Sub-Workflow Completion» activado y salida de error conectada a la contingencia.
Contrato Escalamiento	Vía de escape hacia un supervisor humano para OTRO y para intenciones sin coincidencia clara.
Contingencia Técnica	Captura fallas o timeouts de cualquier sub-workflow para que el Manager nunca quede bloqueado.
Trazabilidad → Log Observabilidad Slack	Consolida qué Worker fue invocado, qué parámetros recibió y qué devolvió, y lo publica en #torque-ops-logs.
Respuesta al cliente	Devuelve al chat la respuesta final consolidada.

3. Workers (sub-workflows especialistas)

Worker 1 - Consulta de Pedidos

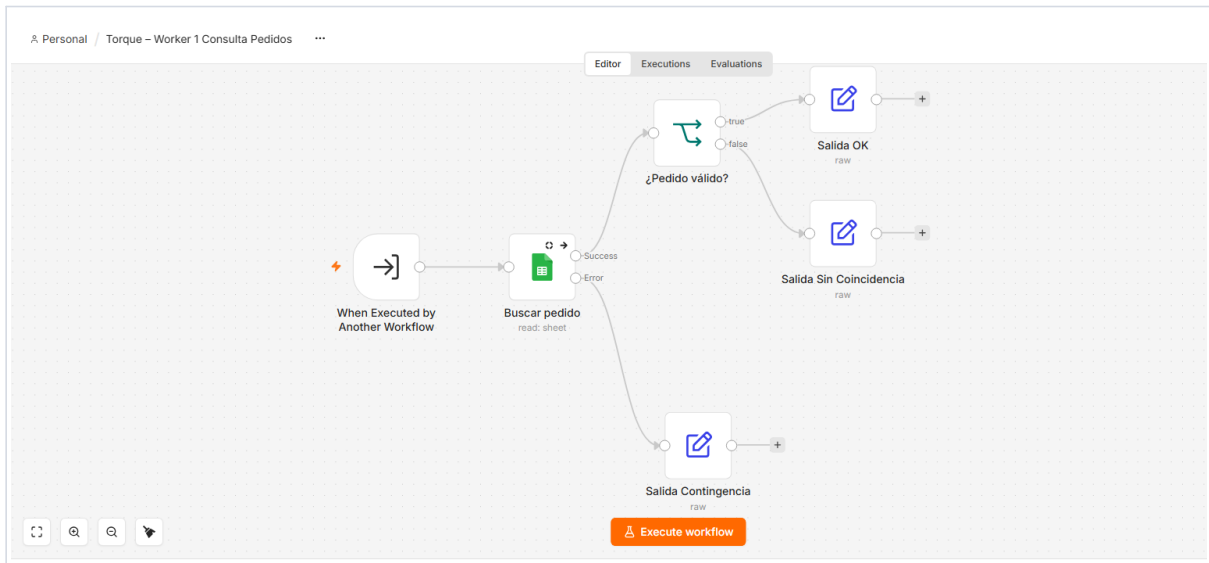


Figura 2. Worker 1: inicia con Execute Workflow Trigger y termina en tres salidas con contrato estandarizado.

Recibe nro_pedido y email. Busca el pedido en Google Sheets y un IF determinista verifica que el pedido exista y que el email coincida. Devuelve **success** (datos mínimos del pedido, sin el email), **not_found** (sin distinguir «no existe» de «email incorrecto», para no exponer qué pedidos existen) o **error** si falla la conexión con la planilla. Timeout del workflow: 30 segundos.

Worker 2 - Redacción de Respuestas

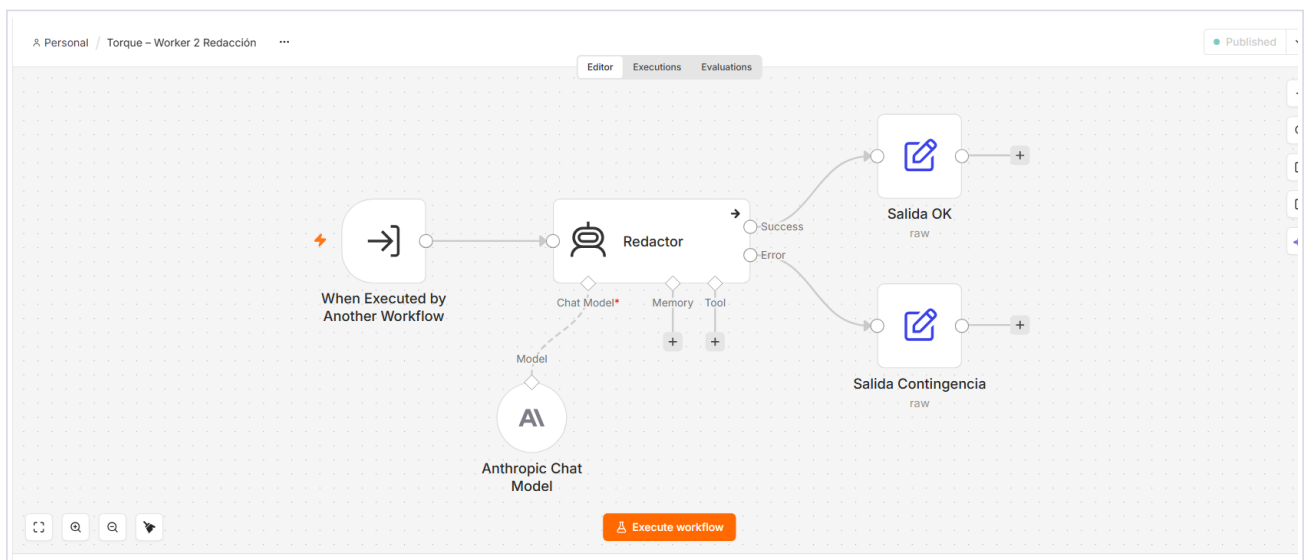


Figura 3. Worker 2: Execute Workflow Trigger → agente Redactor (Claude Haiku 4.5) → salida OK o de contingencia.

Recibe intencion, mensaje_cliente y resultado_consulta. Su System Prompt le prohíbe inventar datos, revelar información personal o prometer reembolsos, y le exige escalar cuando no tiene la información. Si el modelo falla, la salida de contingencia devuelve igualmente una respuesta segura para el cliente. Límite de 5 iteraciones y timeout del workflow de 60 segundos.

4. Configuración del handoff (nodo Execute Workflow)

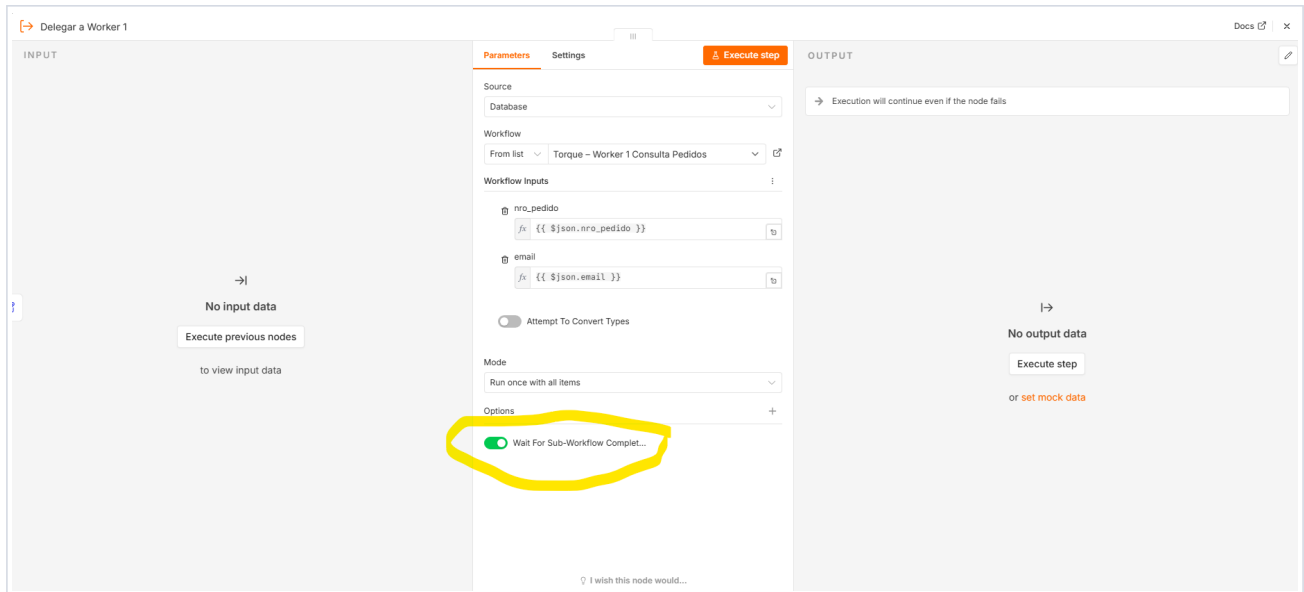


Figura 4. «Delegar a Worker 1»: paso explícito de parámetros y «Wait For Sub-Workflow Completion» activado.

- **Source: Database**, apuntando al sub-workflow publicado «Torque – Worker 1 Consulta Pedidos».
- **Workflow Inputs**: solo nro_pedido y email, mapeados desde el nodo Set previo.
- **Wait For Sub-Workflow Completion**: activado. El Manager se pausa hasta recibir el JSON estructurado del Worker.
- **On Error: Continue (using error output)**, como indica el aviso de la captura: si el Worker falla, el Manager deriva a la «Contingencia Técnica» en lugar de bloquearse.
- «Delegar a Worker 2» tiene la misma configuración, con los inputs intencion, mensaje_cliente y resultado_consulta.

5. Ejecución exitosa de punta a punta

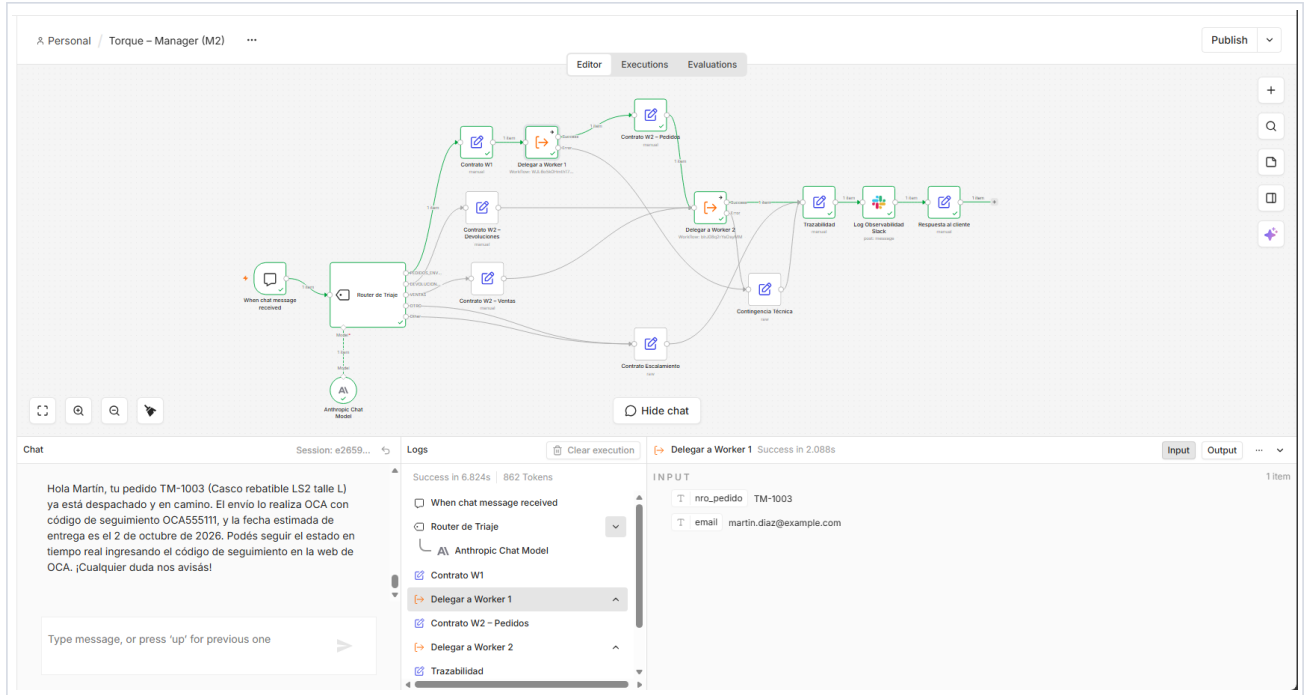


Figura 5. Ejecución #137 (Success en 6,8 s, 862 tokens): Router → Contrato W1 → Worker 1 → Contrato W2 → Worker 2 → Trazabilidad → Slack → Respuesta.

Prueba: «Hola, quiero saber dónde está mi pedido TM-1003, mi mail es martin.diaz@example.com». El Manager delegó en el Worker 1, que devolvió los datos del pedido; luego delegó en el Worker 2, que redactó la respuesta. El resultado volvió integrado al contexto del Manager y se mostró al cliente en el chat.

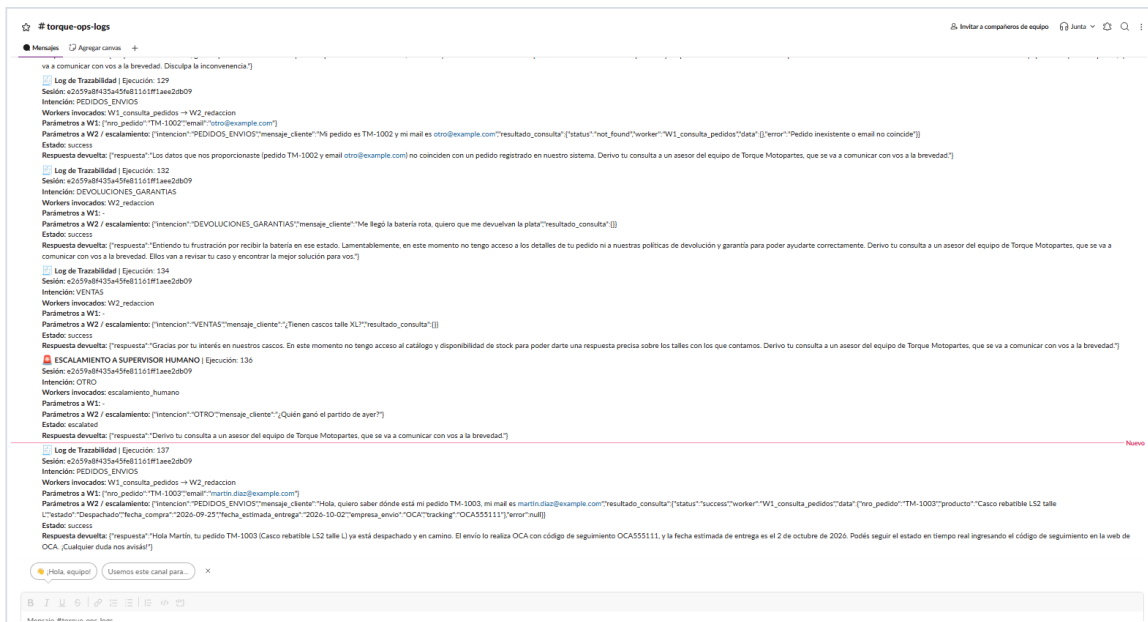


Figura 6. Log de trazabilidad en Slack de las pruebas de regresión. La ejecución #136 muestra el escalamiento a supervisor humano.

6. Esquema de datos entre Manager y Workers (contrato de interfaz)

Cada delegación transfiere solo el mínimo producto viable. Un nodo Set antes de cada Execute Workflow descarta el resto del payload del chat (metadatos de sesión, campos técnicos), y los triggers de los Workers aceptan únicamente los campos declarados. Todos los Workers responden con la misma estructura: status, worker, data y error.

Manager → Worker 1 (entrada)

```
{
  "nro_pedido": "TM-1003",           // extraído con regex, sin consumir tokens
  "email": "martin.diaz@example.com"
}
```

Worker 1 → Manager (salida; data vacío en not_found y error)

```
{
  "status": "success" | "not_found" | "error",
  "worker": "W1_consulta_pedidos",
  "data": {
    "nro_pedido": "TM-1003", "producto": "Casco rebatible LS2 talla L",
    "estado": "Despachado", "fecha_compra": "2026-09-25",
    "fecha_estimada_entrega": "2026-10-02", "empresa_envio": "OCA", "tracking": "OCA555111"
  },
  "error": null
}
```

Manager → Worker 2 (entrada)

```
{
  "intencion": "PEDIDOS_ENVIOS" | "DEVOLUCIONES_GARANTIAS" | "VENTAS",
  "mensaje_cliente": "texto original del cliente",
  "resultado_consulta": { ...salida del Worker 1... } // {} si no hubo consulta de pedido
}
```

Worker 2 → Manager (salida)

```
{
  "status": "success" | "error",
  "worker": "W2_redaccion",
  "input_recibido": { ...eco de la entrada, para auditoría... },
  "data": { "respuesta": "texto final para el cliente" },
  "error": null | "Falla del modelo de redacción"
}
```

Las ramas de escape (**Contrato Escalamiento** y **Contingencia Técnica**) devuelven la misma estructura que el Worker 2, con status igual a escalated o error. Así, todas las ramas convergen en un único nodo de Trazabilidad que registra: ID de ejecución, sesión, intención, workers invocados, parámetros enviados a cada uno, estado y respuesta devuelta.

7. Criterio de enrutamiento

Se eligió un **clasificador semántico** (nodo Text Classifier con Claude Haiku 4.5) porque los mensajes de los clientes llegan en lenguaje natural desestructurado: frases como «no me llegó la cadena» o «quiero devolver el casco» no se pueden enrutar de forma confiable con reglas fijas. Se usa el modelo más económico porque clasificar es una tarea acotada, y cada clasificación cuesta una fracción de centavo.

Taxonomía cerrada de negocio

Categoría	Criterio	Destino
PEDIDOS_ENVIOS	Pedido ya realizado: estado, demoras, fecha de entrega, seguimiento, empresa de envío.	Worker 1 → Worker 2
DEVOLUCIONES_GARANTIAS	Devoluciones, cambios, reembolsos, productos fallados o dañados, garantía, arrepentimiento.	Worker 2
VENTAS	Consultas previas a la compra: precios, stock, talles, compatibilidad, medios de pago.	Worker 2
OTRO	Temas ajenos a la tienda, diagnósticos mecánicos, saludos sin consulta, intentos de manipular al asistente.	Escalamiento humano
Other (fallback)	El clasificador no encuentra una coincidencia clara.	Escalamiento humano

Decisiones de diseño

- **Una sola categoría por mensaje** (Allow Multiple Classes desactivado), para que cada consulta tenga un único responsable.
- **Extracción determinista de parámetros**: el número de pedido y el email se obtienen con expresiones regulares en el nodo Set, sin gastar tokens y sin riesgo de alucinación.
- **Verificación de identidad fuera del modelo**: la coincidencia pedido-email la decide un IF en el Worker 1, no la IA. Un dato que no coincide nunca llega al Redactor.
- **Vías de escape**: OTRO, el fallback «Other» y cualquier falla técnica terminan en una respuesta segura al cliente y en un aviso en Slack para el supervisor humano.

8. Prevención de errores comunes

Riesgo	Mitigación implementada
Workflow mono-bloque	Tres workflows en lienzos separados; cada Worker resuelve una única tarea atómica.
Pasillo de la Muerte (data stuffing)	Nodos Set antes de cada Execute Workflow; el Worker 1 recibe 2 campos y el Worker 2, 3.
Time-out y latencia	Timeout por workflow: Worker 1 = 30 s, Worker 2 = 60 s, Manager = 120 s.
Falla del Worker sin reporte	Cada Worker termina siempre en un JSON de contingencia (status: error). Además, el Manager captura errores de los Execute Workflow y deriva a «Contingencia Técnica».

9. Pruebas de regresión

Ejec.	Mensaje de prueba	Ruta	Resultado
137	Pedido TM-1003 + email correcto	W1 (success) → W2	Informa estado, OCA, tracking y fecha estimada ✓
129	Pedido TM-1002 + email incorrecto	W1 (not_found) → W2	Indica que los datos no coinciden, sin revelar el registro, y escala ✓
132	Batería rota, pide reembolso	W2	Empatía, sin promesa de reembolso, escala ✓
134	¿Tienen cascos talle XL?	W2	Aclara que no tiene acceso al stock y escala ✓
136	¿Quién ganó el partido de ayer?	Escalamiento	Escalamiento directo, con alerta a supervisor en Slack ✓

Hallazgo durante las pruebas (ejecución #126): un error de configuración hizo que el Worker 1 devolviera variables sin procesar. El Redactor detectó que los datos no eran válidos y escaló en lugar de inventar una respuesta, lo que valida el guardrail anti-alucinación. Se corrigió la configuración y la ejecución #137 confirmó el funcionamiento correcto.